WEB APPLICATION VULNERABILITY ASSESSMENT REPORT

Target Environment: Damn Vulnerable Web Application (DVWA)

Assessment Type: Web Application Penetration Testing
Prepared Date: 16 May 2026

Prepared by

Name :Rebaka Rahman Jannat Shammi

ID: 2022200000040

Section:CADS008

1. Executive Summary

This report presents the results of a controlled vulnerability assessment performed against the Damn Vulnerable Web Application (DVWA) environment. The objective of the assessment was to identify common web application security weaknesses and validate exploitation techniques in a laboratory environment. Multiple vulnerabilities including SQL Injection, Cross Site Scripting (XSS), File Inclusion, Command Execution, Brute Force, and File Upload weaknesses were identified during testing.

2. Scope of Assessment

- Authentication Mechanisms
- Input Validation Testing
- Database Security Validation
- Command Execution Testing
- File Upload and File Inclusion Functionalities
- Cross Site Scripting (XSS) Testing

3. Assessment Methodology

- Manual penetration testing
- HTTP request interception using Burp Suite
- Payload injection and parameter manipulation
- Source code review and response analysis
- Database enumeration techniques
- Validation of exploitation results

4. Risk Severity Classification

Severity	Description
Critical	Direct compromise of server, application, or sensitive data.
High	Serious security weakness with major impact.
Medium	Moderate security issue requiring remediation.
Low	Limited impact vulnerability.

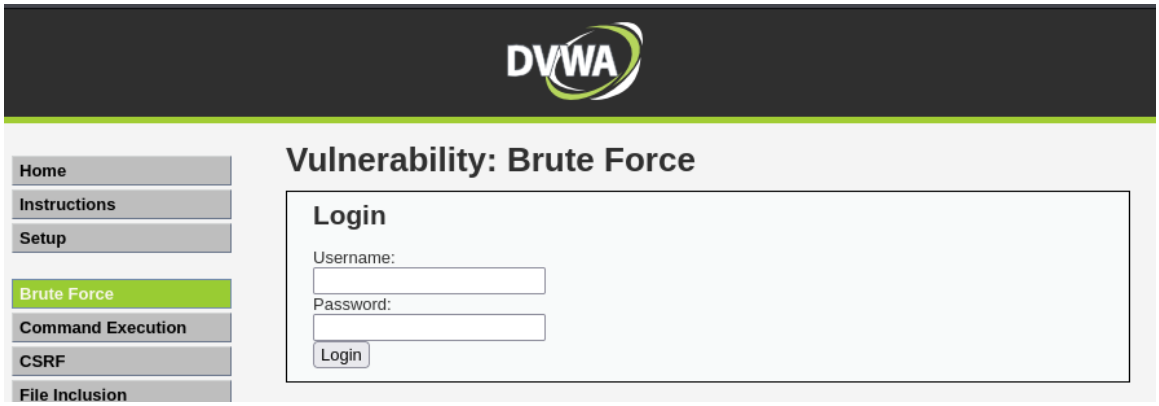
5.1 Brute Force Vulnerability

Severity	High
Description	The login mechanism did not implement account lockout or rate limiting protections. Burp Suite Intruder was used to automate authentication attempts.
Impact	Unauthorized access through credential guessing attacks.
Recommendation	Implement account lockout, CAPTCHA, and multi-factor authentication.
Status	Confirmed

Proof of Concept / Evidence:

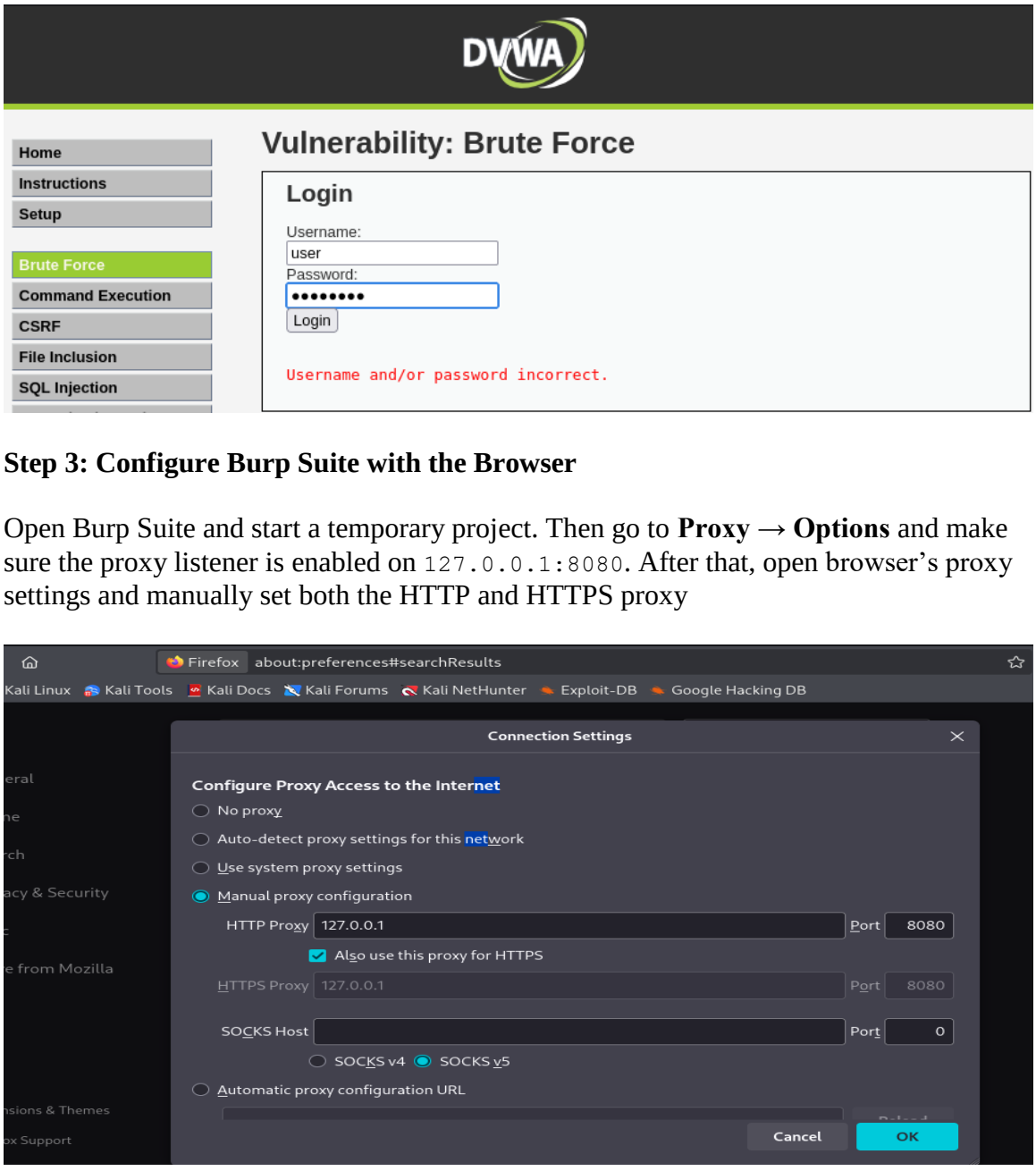
Step 1:

Open the Brute Force vulnerability page from DVWA.



Step 2: Test the Login Form with Random and Common Credentials

In this phase, the login form is tested by entering a variety of random usernames and passwords, including commonly used default credentials. The goal is to observe how the application handles unsuccessful login attempts and to identify any clues about the underlying authentication mechanism.



Step 3: Configure Burp Suite with the Browser

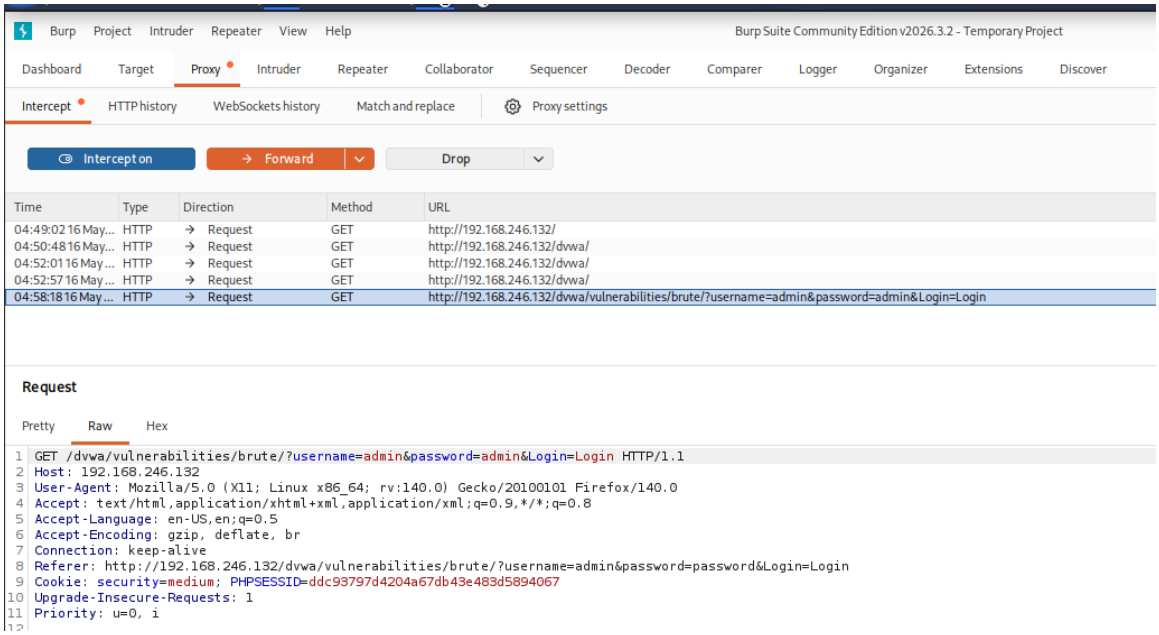
Open Burp Suite and start a temporary project. Then go to **Proxy → Options** and make sure the proxy listener is enabled on `127.0.0.1:8080`. After that, open browser's proxy settings and manually set both the HTTP and HTTPS proxy

Figure 1.1: Evidence related to Brute Force Vulnerability

Step 4: Load DVWA Brute Force Page and Capture Login Request

Open the Damn Vulnerable Web Application (DVWA) and go to the **Brute Force** page. Enter any username and password, then click login. Since Burp Suite is already configured as a proxy, the request will be intercepted automatically.

Check **Proxy → Intercept** in Burp Suite to view the captured login request containing the username and password parameters. This allows you to analyze or modify the request for further testing.

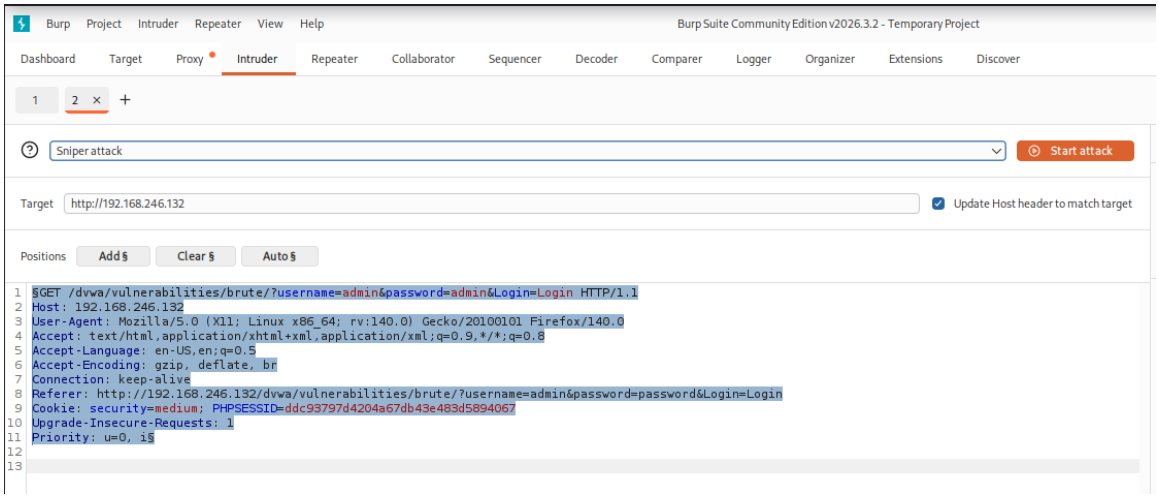


Step 5: Send Request to Intruder for Brute Force

In Burp Suite, right-click the captured login request from the Proxy tab and select **"Send to Intruder."**

This will open the request in the Intruder tool, where you can configure positions for the **username/password fields** and prepare them for automated testing.

This step is used to automate repeated login attempts against the Damn Vulnerable Web Application (DVWA) Brute Force page for testing authentication security.

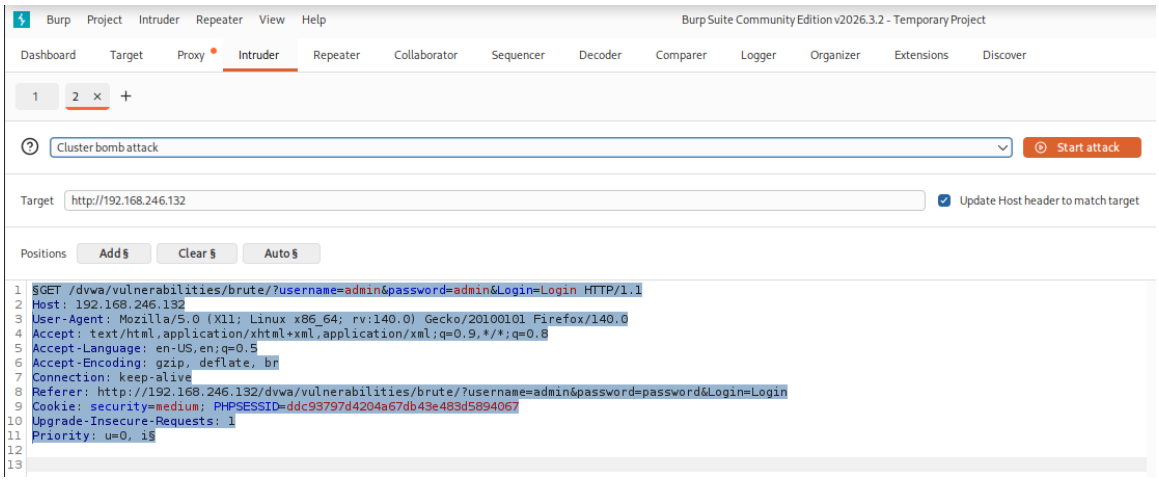


Step 6: Configure Payload Positions and Select Cluster Bomb

In Burp Suite Intruder, first highlight the input fields (such as **username** and **password**) as payload positions.

Then go to the **Attack Type** section and select **Cluster Bomb**.

This attack type is used when there are multiple input fields because it tests all possible combinations of payloads for each selected position. It is commonly used in brute force testing on the Damn Vulnerable Web Application (DVWA) login form.



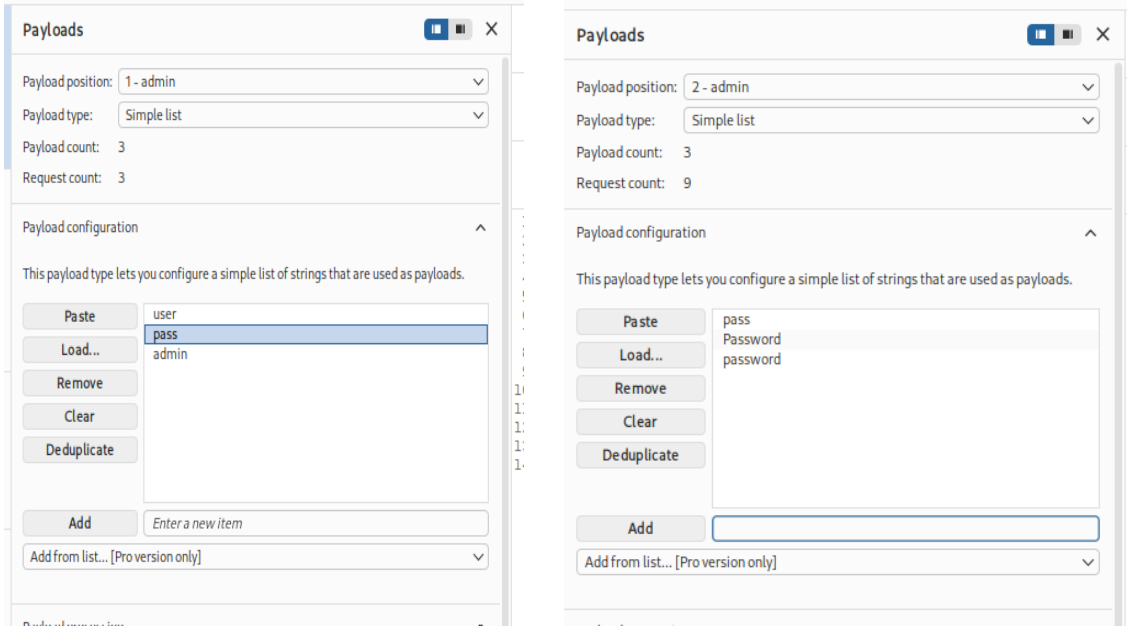
Step 7: Add Payloads for 2 Parameters

In Burp Suite Intruder, go to **Payloads** and set two payload lists for the selected fields (username and password).

Example:

- Username list: `user, pass, admin`
- Password list: `pass, Password, password`

These values will be combined and tested automatically in the Damn Vulnerable Web Application (DVWA) login page.

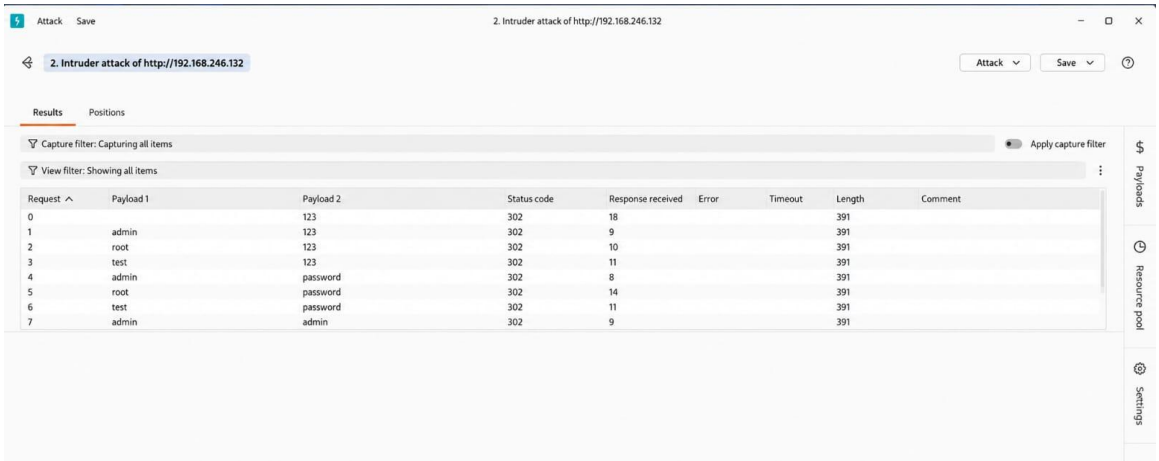


Step 8: Start Attack and Analyze Responses

In Burp Suite Intruder, click **Start Attack** and monitor the results.

Check the **Status code** and **Response Length** for each request. If one response looks different from the others, it may indicate valid credentials.

Test that username and password manually on the Damn Vulnerable Web Application (DVWA) login page to confirm the result.



Note:

The overall procedure remains the same for **Low, Medium, and High security levels** in Damn Vulnerable Web Application (DVWA). However, as the security level increases, additional filtering, validation, or protection mechanisms may be applied, which can affect the effectiveness of payloads and may require adjusted or more advanced payload lists during testing in Burp Suite.

5.2 Command Execution

Severity	Critical
Description	Unsanitized input allowed execution of operating system commands on the server.
Impact	Remote command execution may lead to complete server compromise.
Recommendation	Validate input and avoid direct shell execution functions.
Status	Confirmed

Proof of Concept / Evidence:

Step 1:

Open problem in dvwa

Step 2: Review Source Code

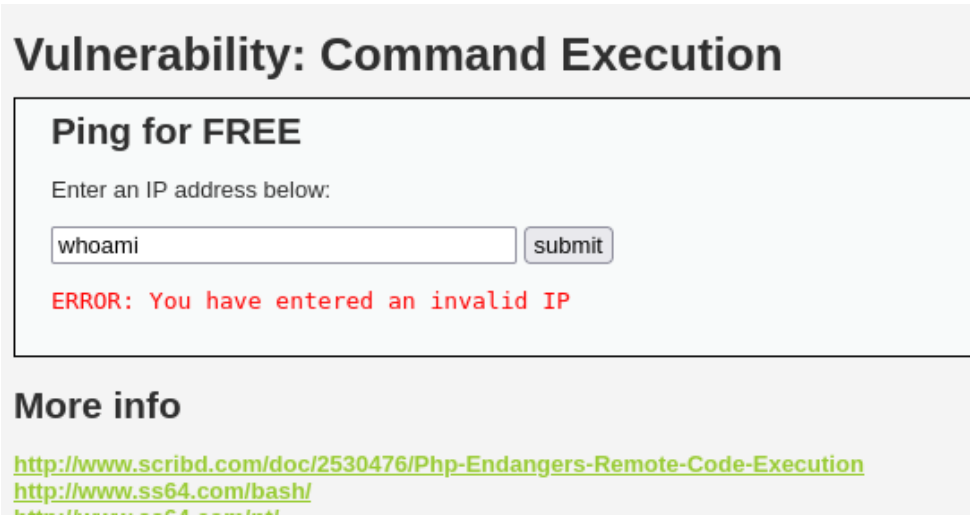
Check the source code to see how the application executes system commands. This helps identify how user input is processed and which command structure is used in the backend, including the possible operating system behavior.

Command Execution Source

```php
<?php

if( isset( $_POST[ 'submit' ] ) ) {

    $target = $_REQUEST["ip"];

    $target = stripslashes( $target );


    // Split the IP into 4 octects
    $octet = explode(".", $target);

    // Check IF each octet is an integer
    if ((is_numeric($octet[0])) && (is_numeric($octet[1])) && (is_numeric($octet[2])) && (is_numeric($octet[3])) && (sizeof($octet) == 4)  ) {

    // If all 4 octets are int's put the IP back together.
    $target = $octet[0].'.'.$octet[1].'.'.$octet[2].'.'.$octet[3];


        // Determine OS and execute the ping command.
        if (stristr(php_uname('s'), 'Windows NT')) {

            $cmd = shell_exec( 'ping  ' . $target );
            echo '<pre>'.$cmd.'</pre>';

        } else {

            $cmd = shell_exec( 'ping  -c 3 ' . $target );
            echo '<pre>'.$cmd.'</pre>';

        }

    }

    else {
        echo '<pre>ERROR: You have entered an invalid IP</pre>';
    }


}
```

Here the server is Windows.

Step 3: So try to execute the Windows command..



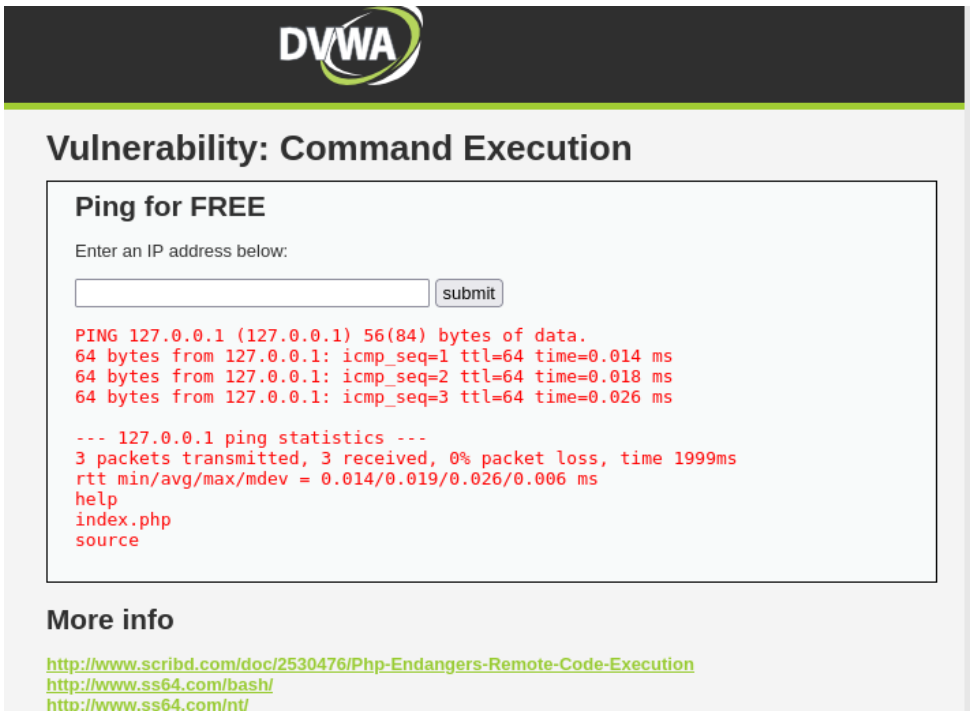
But not working. Because back-end queries do not match with this command. So to fix this problem, we need to show source code .

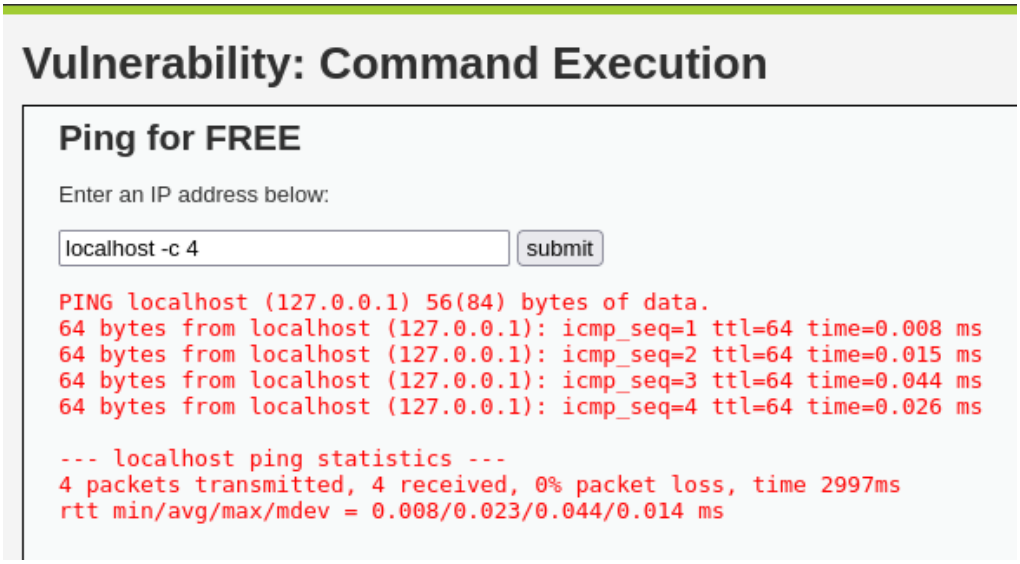
Command Execution Source

```php
<?php

if( isset( $_POST[ 'submit' ] ) ) {

    $target = $_REQUEST["ip"];

    $target = stripslashes( $target );


    // Split the IP into 4 octects
    $octet = explode(".", $target);

    // Check IF each octet is an integer
    if ((is_numeric($octet[0])) && (is_numeric($octet[1])) && (is_numeric($octet[2])) && (is_numeric($octet[3])) && (sizeof($octet) == 4)  ) {

    // If all 4 octets are int's put the IP back together.
    $target = $octet[0].'.'.$octet[1].'.'.$octet[2].'.'.$octet[3];


        // Determine OS and execute the ping command.
        if (stristr(php_uname('s'), 'Windows NT')) {

            $cmd = shell_exec( 'ping  ' . $target );
            echo '<pre>'.$cmd.'</pre>';

        } else {

            $cmd = shell_exec( 'ping  -c 3 ' . $target );
            echo '<pre>'.$cmd.'</pre>';

        }

    }

    else {
        echo '<pre>ERROR: You have entered an invalid IP</pre>';
    }


}
```

Here we see back-end query is localhost and localhost -c 4 so if we try execute any command first we need to use locahost command.

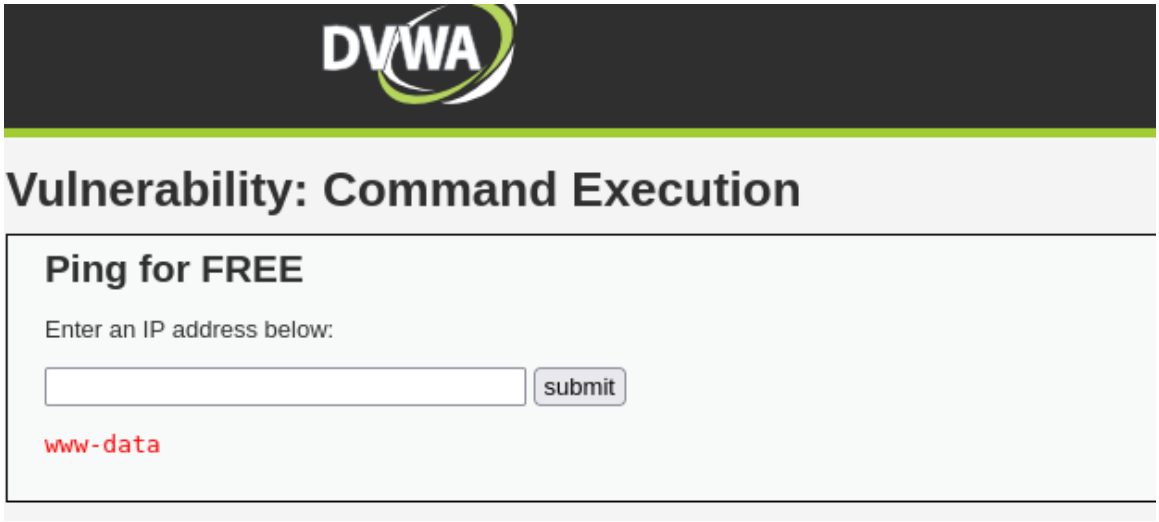
Step-4: Try to execute a command with localhost.



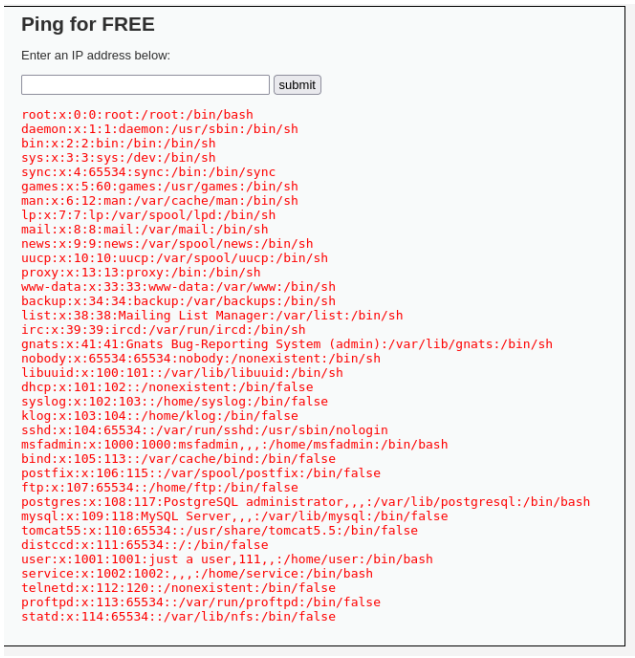


Step 5: Test Input Isolation and Command Handling

Test how the application behaves when previous parameters or inputs are ignored or modified. Observe whether the backend properly isolates user input from system commands and whether input validation prevents unintended command execution. This helps evaluate the strength of command injection protections.



Yes, it is working ! Here I use ;whoami command and show the result. But why show result ? Because here use ; for comment out all previous commands. Now you can try another comment out way like &,|, || etc.



wow found all the directories. Here I use ; cat *etc/passwd* command .

NB: In high , medium level process are same but every category have whitelist some key . So every level you try to see source code and note down whitelist key and try to inject a command without that's keys.

5.3 File Inclusion

Severity	High
Description	The application accepted user-controlled file path parameters, enabling Local File Inclusion attacks.
Impact	Sensitive files and server information may be disclosed.
Recommendation	Use strict path validation and allow only approved files.
Status	Confirmed

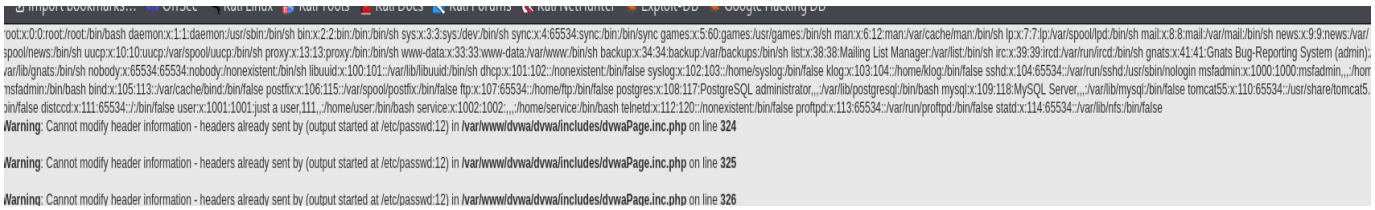
Proof of Concept / Evidence:

Step-1: Open the File Inclusion vulnerability page in DVWA..

Step-2: Flow the url and find your input field.

and input field, I mean parameter is (page=). So here we try to inject our command.

We need to file inclusion for this server . We know every sensitive file is included in the *etc*/passwd directory, so try it.

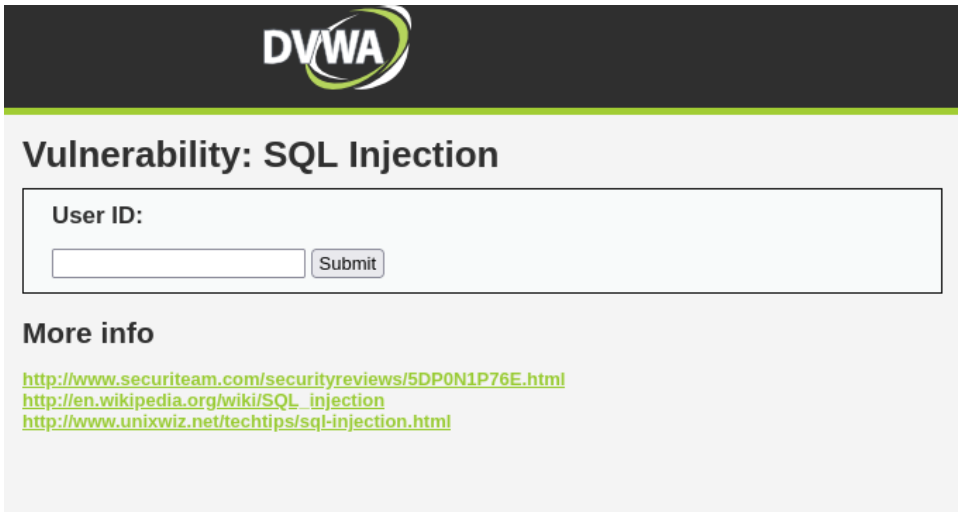


Yes, it's working. But for the advanced levels, it does not work to use this easy process. Ok, try to see a high-level problem.

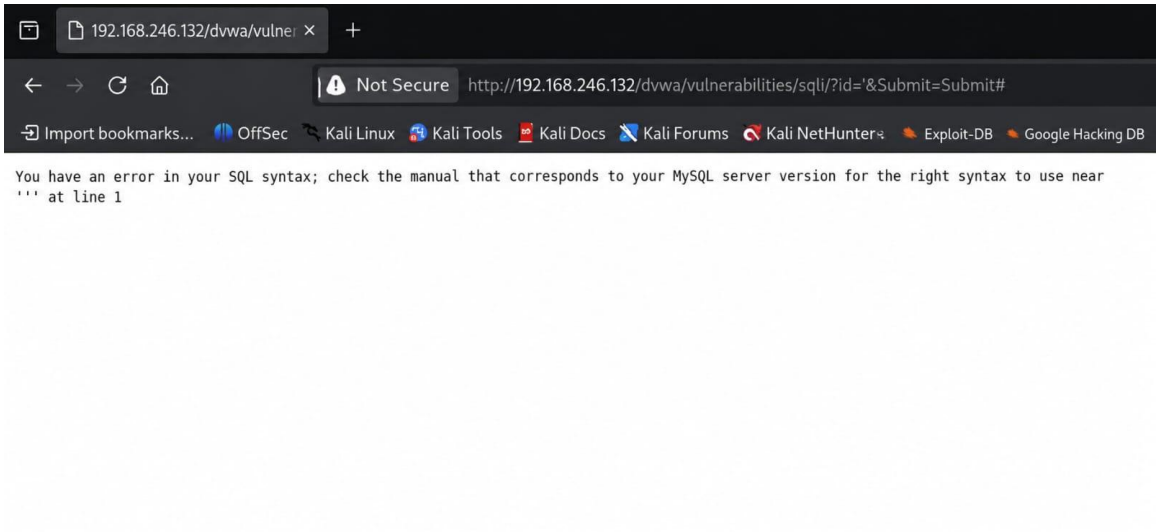
I complete this section letter.

Problem 4: SQL injection

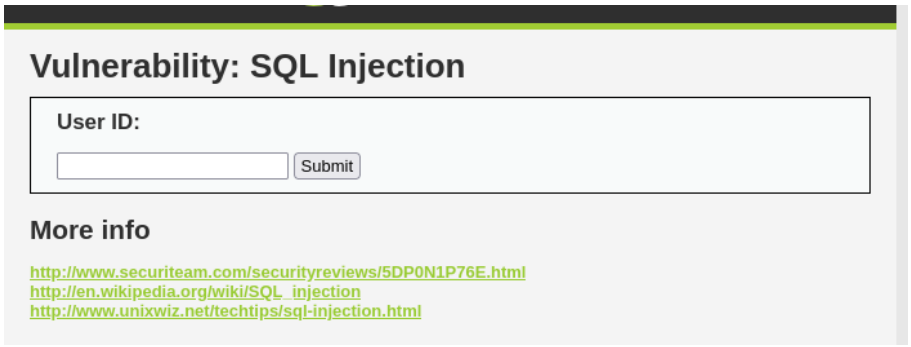
Step 1: Open the SQL Injection vulnerability page in DVWA.



Step-2: Check this input field is vulnerable or not. So we use ‘ for show vulnerable message.

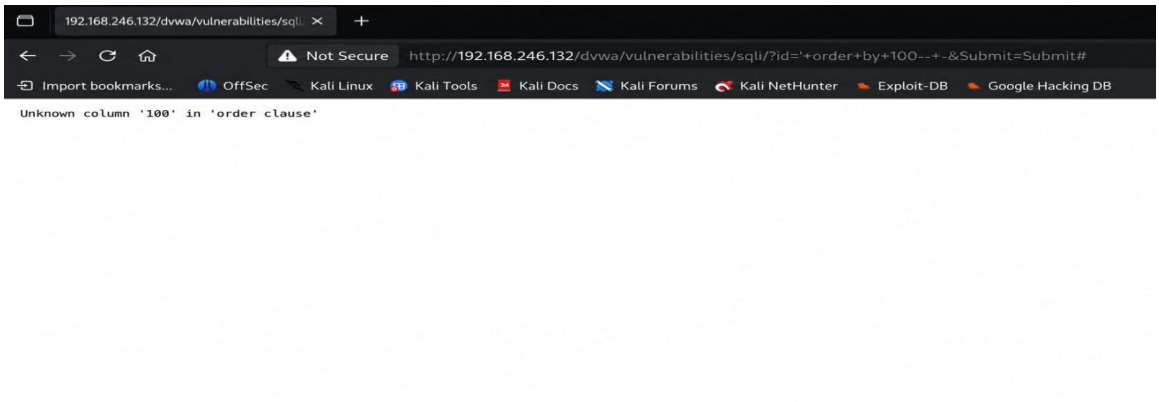


Step-3: Bypass the syntax error using SQL comments.

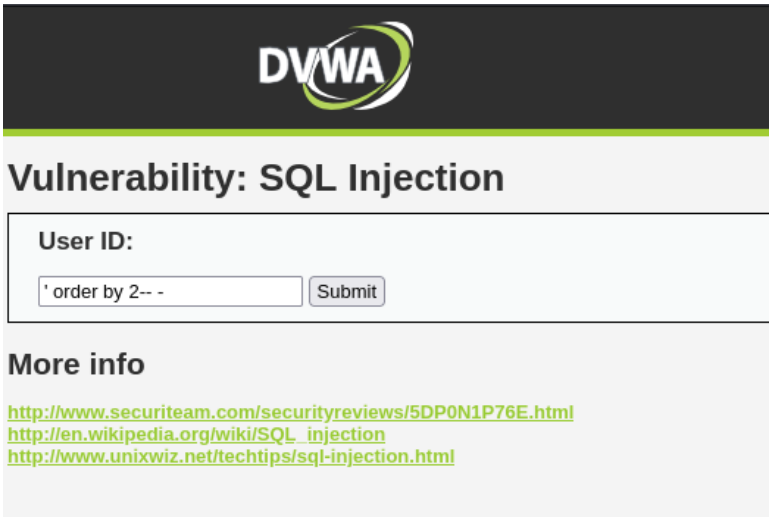


No vulnerable message because problem is fixed. Here I use – - for comment out all next query.

Step-4: Now find out how many column have this database.



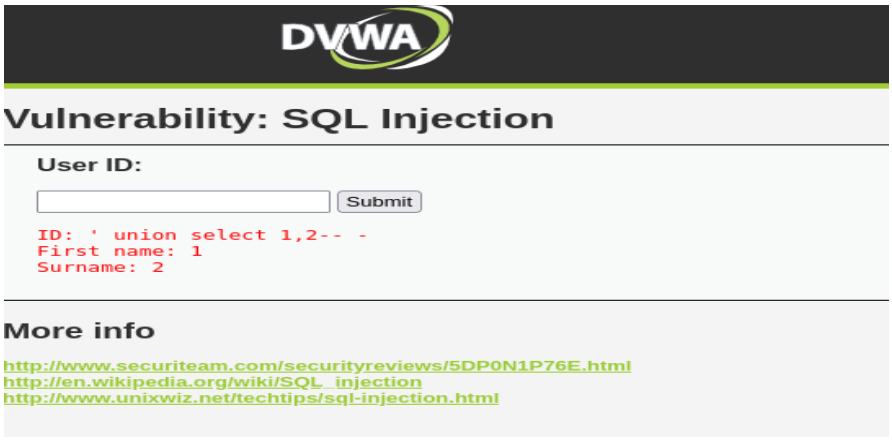
Use ‘ order by 100-- - and result is unknown column 100. That’s mean this database table have not exist 100 column. So minimize column number and find no error message.



shows no error, indicating that the table contains 2 columns.

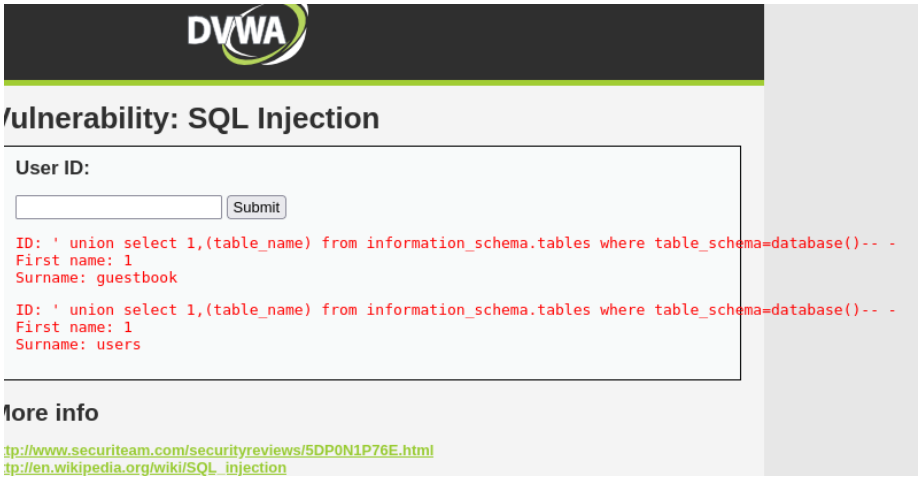
Step-5: Identify vulnerable columns using UNION SELECT.

‘ union select 1,2-- -



Here 1, 2 both column is vulnerable.

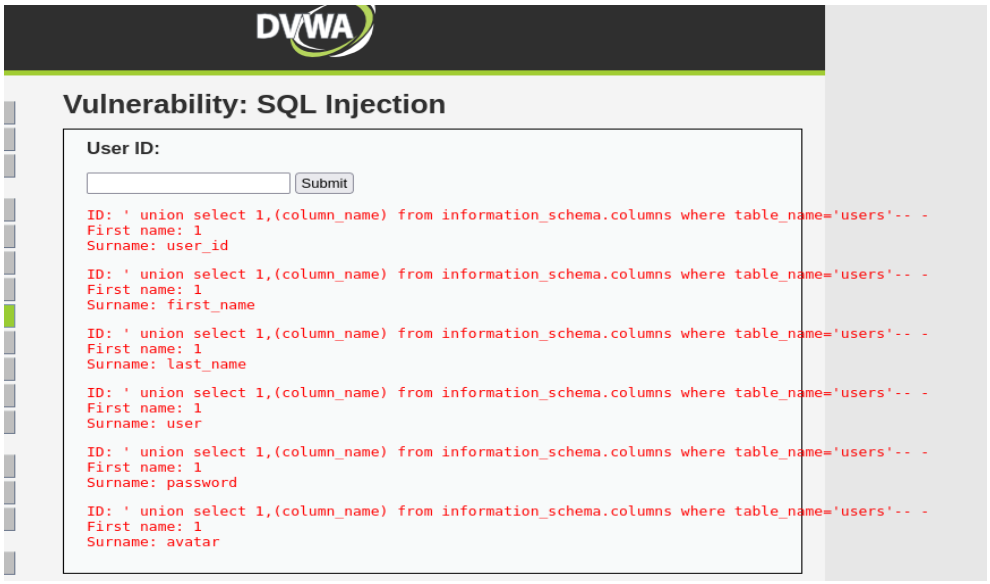
Step-6: Next step is retrieve table name from database.



Command is : ' union select 1,(table_name) from information_schema.tables where table_schema=database()-- -

Here 2 table name found first is gestbook and second is users. Both table have information but users column have high possibility to have user login information.

Step-7: In this step we try to discover column name from users table.



Here using command is : ' union select 1,(column_name) from information_schema.columns where table_name='users'-- -

This command retrieve all column name from users table. But I hope user and password column have login credential.

Step-8: Now try to retrieve data from user and password column .

Using command is: ' union select 1,group_concat(user,'::',password) from users-- -

Here we see user login credential.

Username = admin

Password = 5f4dcc3b5aa765d61d8327deb882cf99

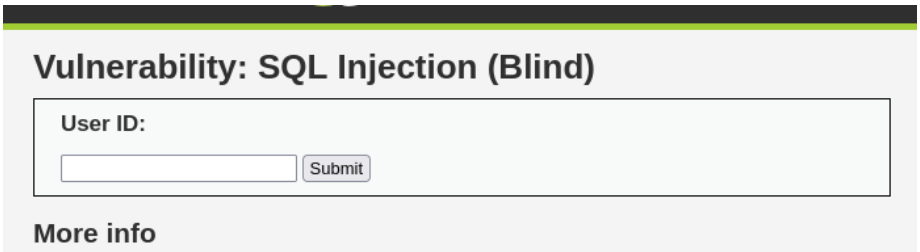
5.5 Blind SQL Injection

Severity	Critical
Description	Blind SQL Injection was confirmed using boolean-based conditions and database enumeration techniques.
Impact	Hidden database information can be extracted without visible SQL errors.
Recommendation	Apply proper input sanitization and secure query handling.
Status	Confirmed

Proof of Concept / Evidence:

Step-1: Start problem environment .

Step-2: Test for SQL Injection using:



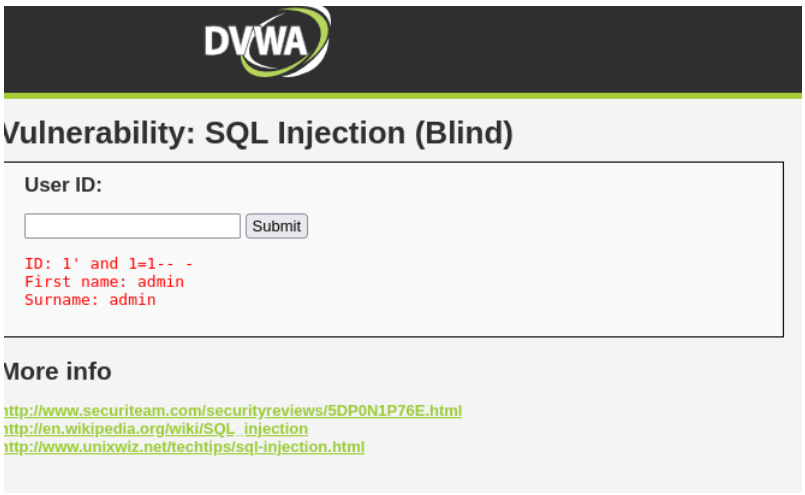
No visible SQL error appears because the application uses Blind SQL Injection handling.

How we can sure it lets see in next step.

Step-3: Verify Blind SQL Injection behavior.

Use a true condition:

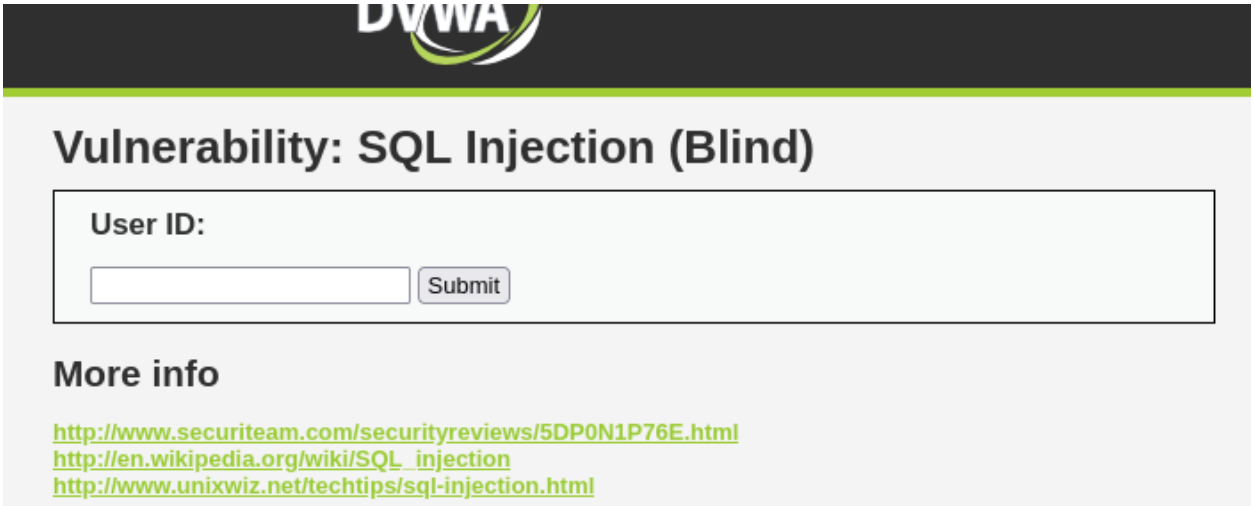
1' and 1=1-- -



Show valid page !

But when use invalid statement

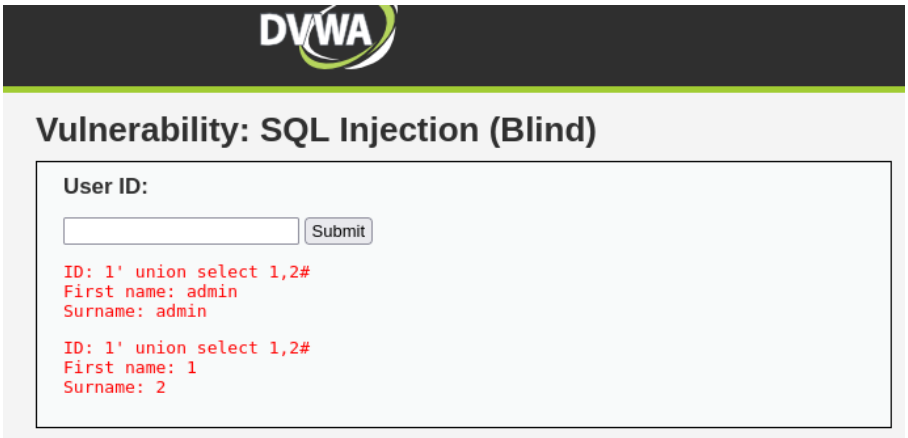
1' and 1=1-- -



Not show valid page ! So we can say this site are blind sql vulnerable.

Step-4: Identify total column number from database.

Command is : 1' order by 2#

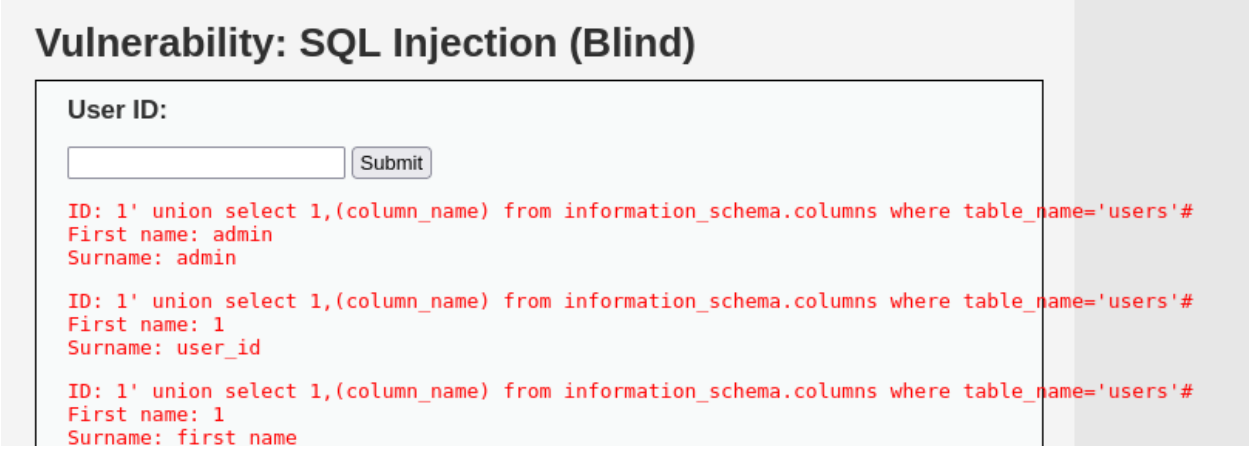


2 column found!

Here both column is vulnerable

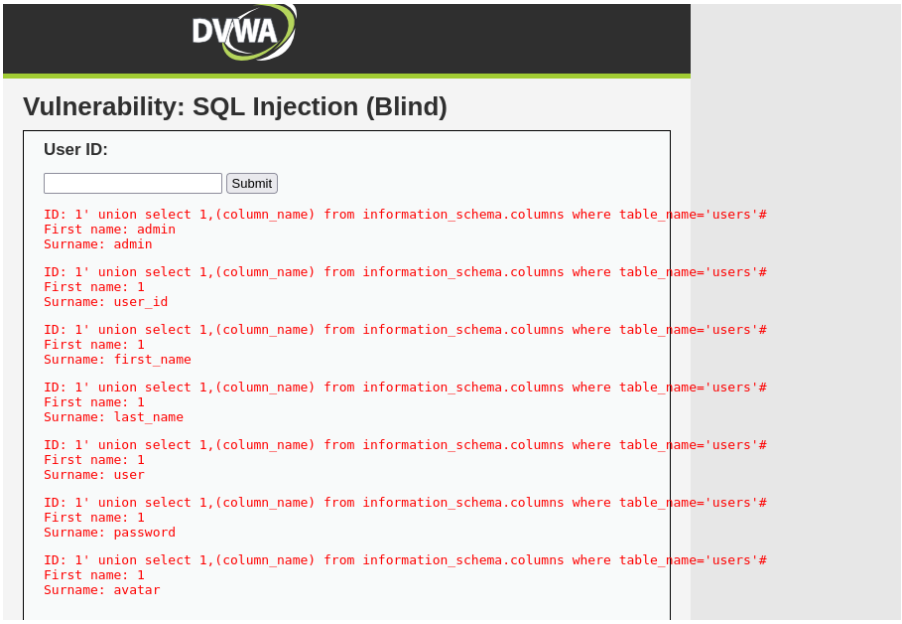
Step-5: Try to retrieve table name.

Using command is : 1' union select 1,(table_name) from information_schema.tables where table_schema=database()#



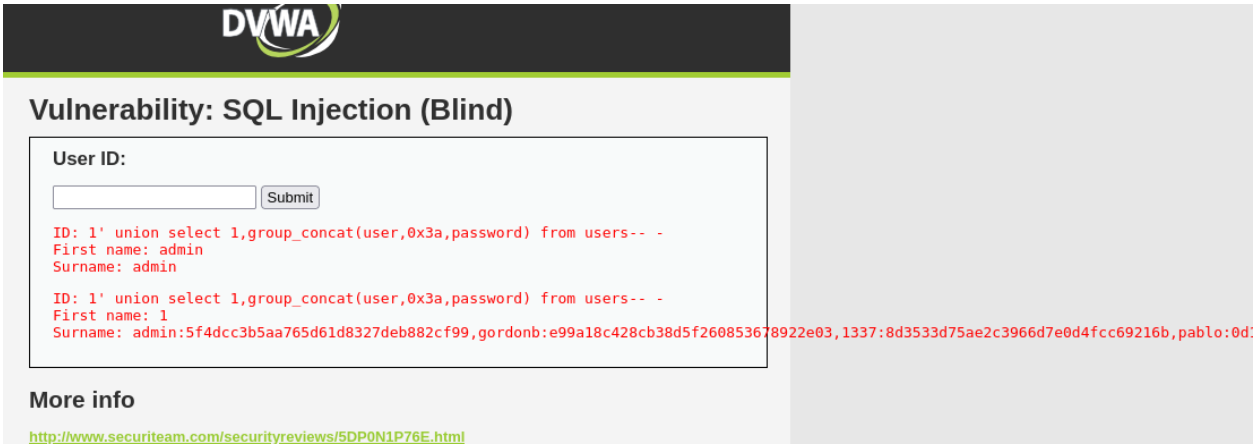
Step-7: Retrieve column names from the users table.

Command is : union select 1,(column_name) from information_schema.columns where table_name='users'



Step-8: last step try to retrieve user login credential.

Using command : union select 1,group_concat(user,'::',password) from users



User name : admin

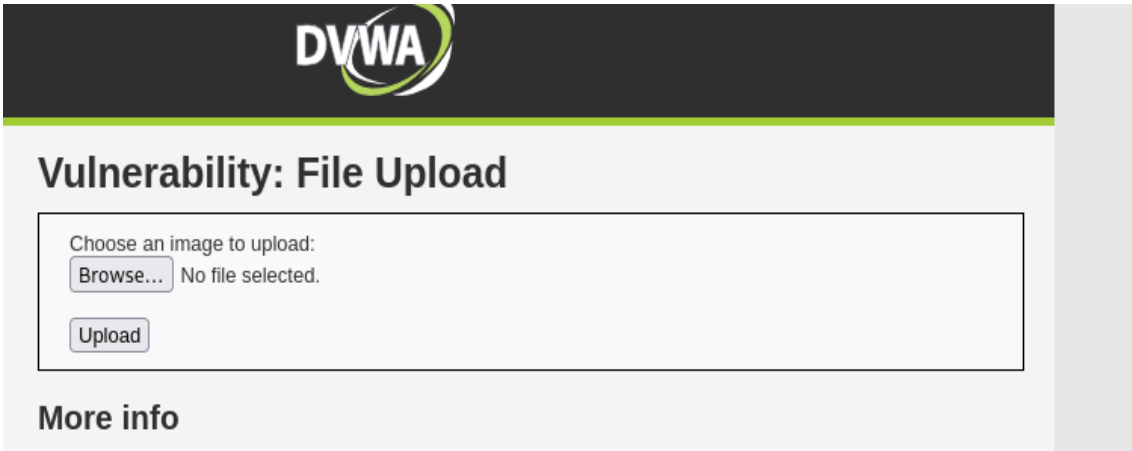
Password: 5f4dcc3b5aa765d61d8327deb882cf99

5.6 File Upload Vulnerability

Severity	High
Description	The upload functionality failed to properly validate uploaded files.
Impact	Malicious files may lead to remote code execution.
Recommendation	Restrict executable uploads and validate MIME types.
Status	Confirmed

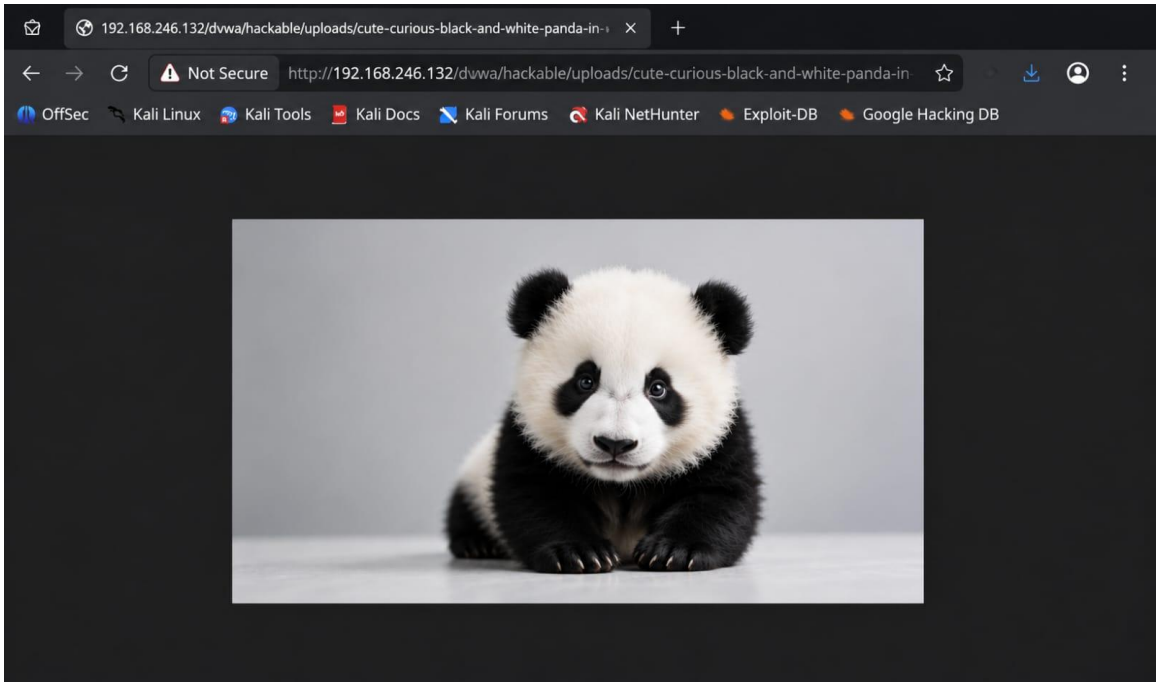
Proof of Concept / Evidence:

Step-1: Open the File Upload vulnerability page.



Step-2: Select a file from the local machine using the upload option and submit it to the server.

In this test, an image file was uploaded successfully.



Boom ! My shell is working.

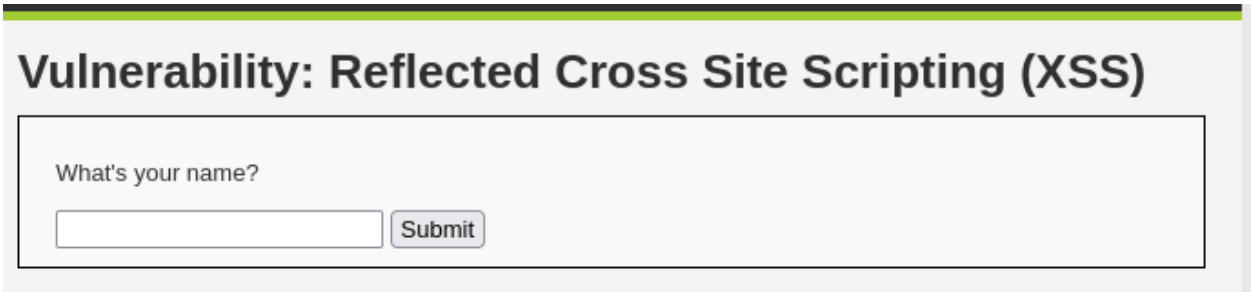
But when try to upload file in high or medium level vulnerabilities then you need to use some bypass technique. Ike you try to include shell code in image file and save this file name like file.php.jpg and try to upload this file. I hope it will work.

5.7 Reflected Cross Site Scripting (XSS)

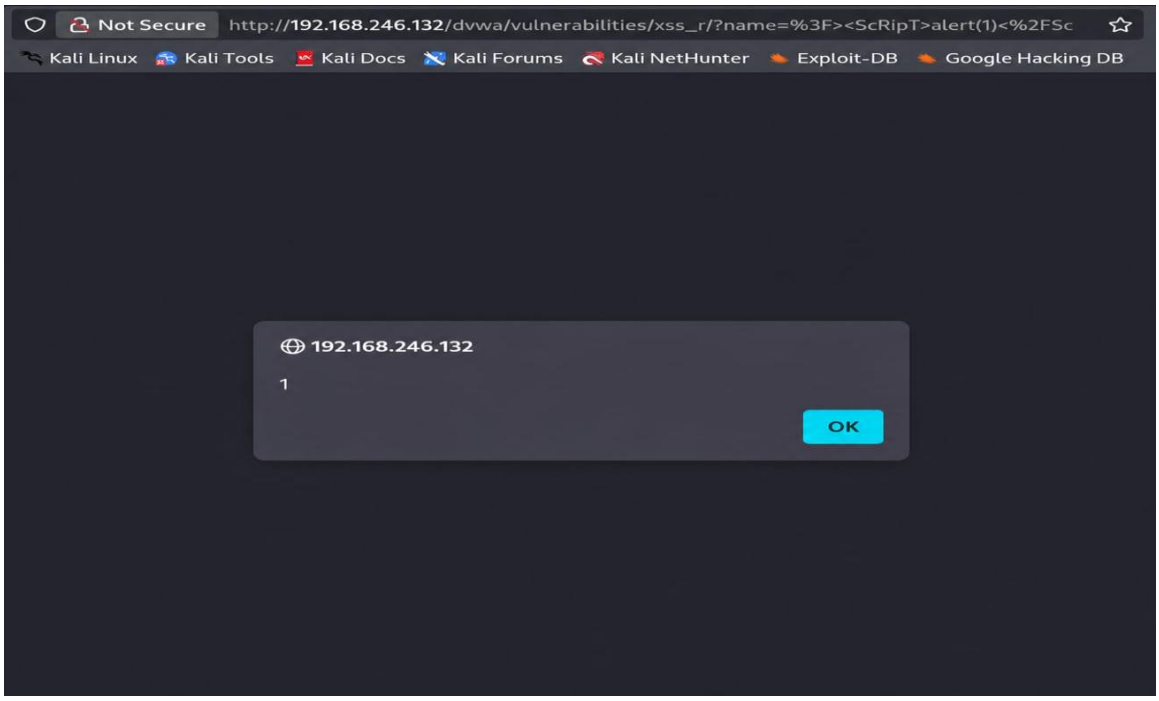
Severity	Medium
Description	JavaScript payloads executed through reflected user input fields.
Impact	Attackers may steal session tokens or perform client-side attacks.
Recommendation	Sanitize and encode user-controlled output.
Status	Confirmed

Proof of Concept / Evidence:

Step-1: Open the Reflected XSS vulnerability page.

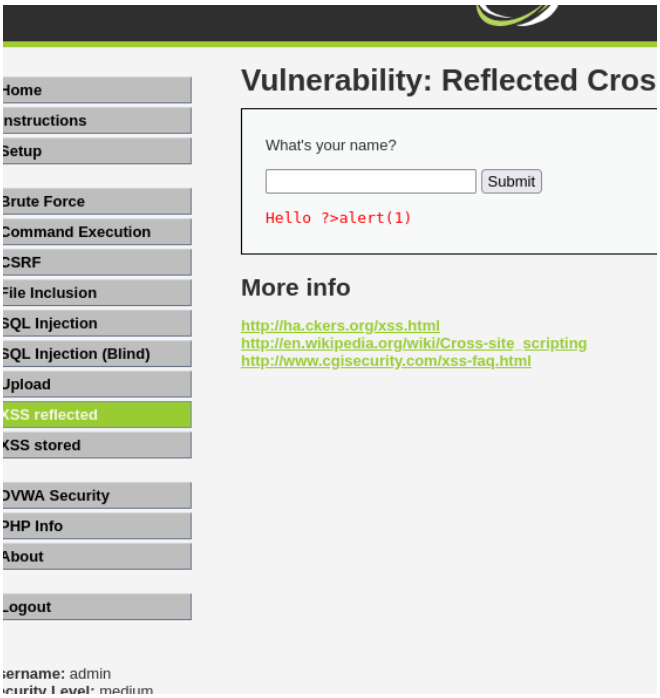


Step-2: Inject a basic XSS payload.



Yes it's working. Here payload is ?><script>alert(1)</script>

ok lets go medium level !



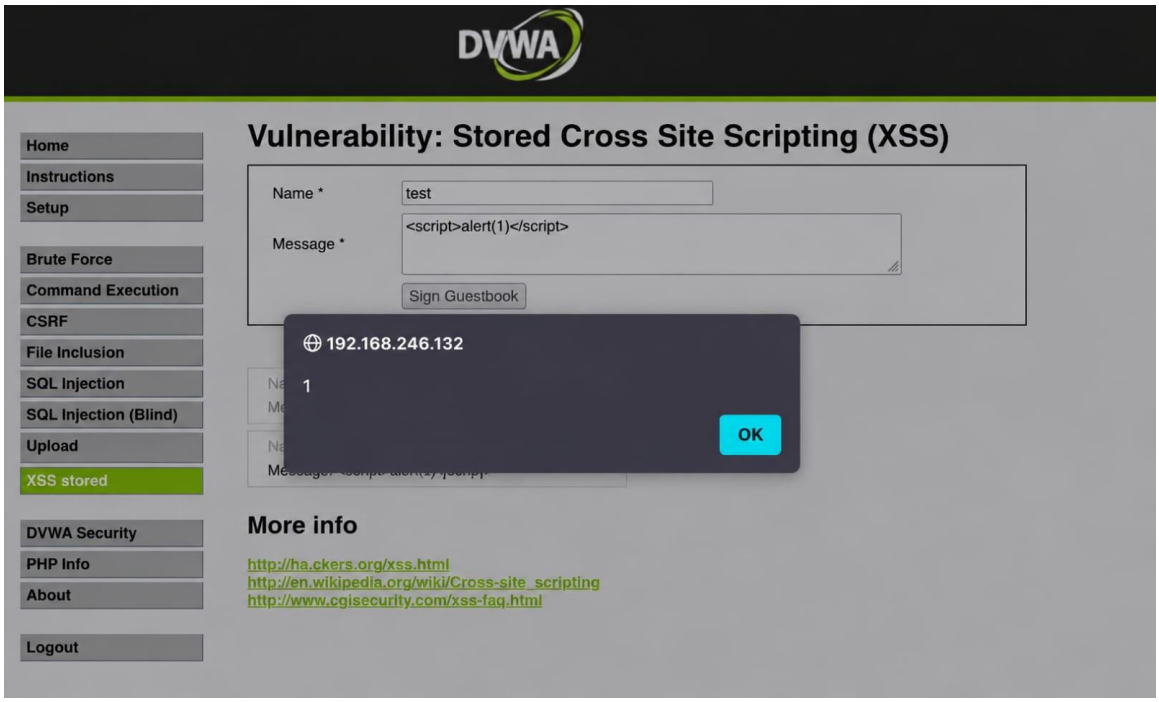
Same payload input here but not working.

But why ? Because this input field filter script tag. For check it you need to see source code.

Reflected XSS Source

```php
<?php

if(!array_key_exists ("name", $_GET) || $_GET['name'] == NULL || $_GET['name'] == ''){

 $isempty = true;

} else {

 echo '<pre>';
 echo 'Hello ' . str_replace('<script>', '', $_GET['name']);
 echo '</pre>';

}

?>
```

Here we can see script tag are replaced. So now we try to bypass script tag and inject.



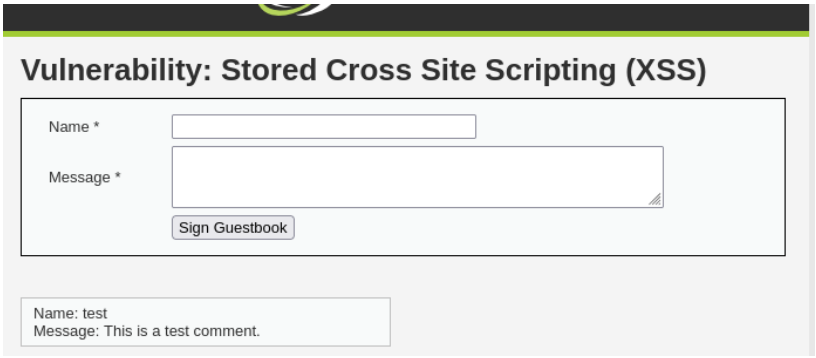
Now this payload is working. Because we use to bypass technique for xss script tag. Here we use ?><ScRipT>alert(1)</ScRiPt> tag. This payload convert by case sensitive.

5.8 Stored Cross Site Scripting (XSS)

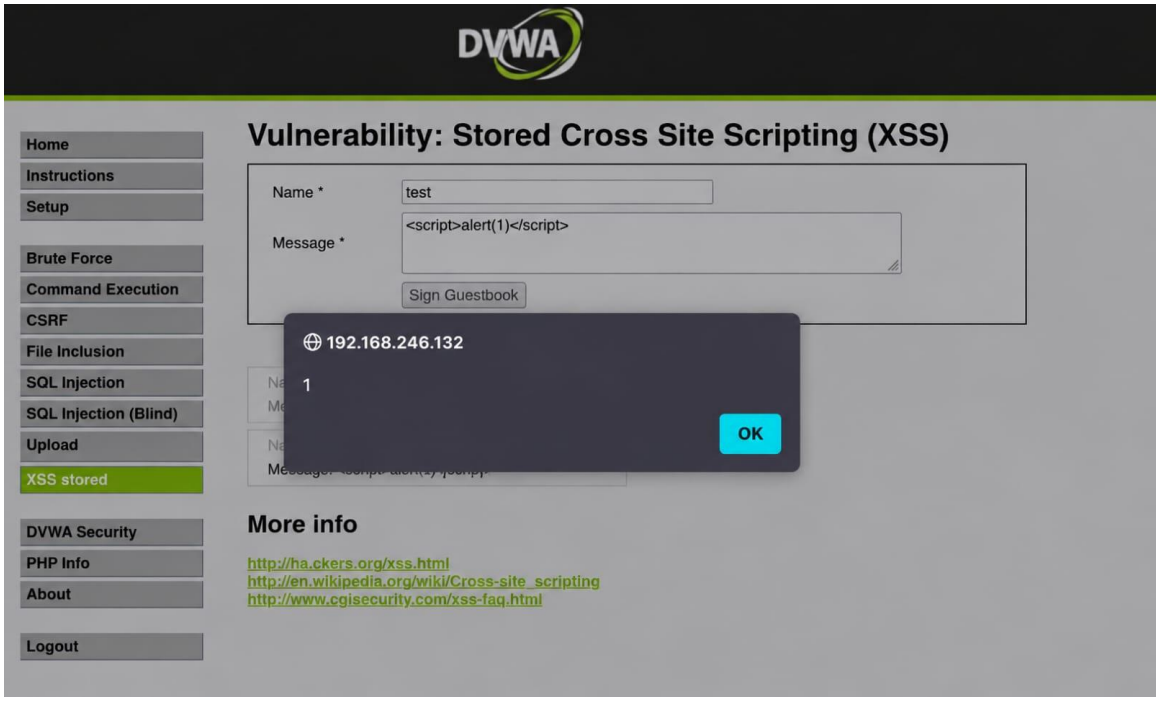
Severity	High
Description	Malicious JavaScript payloads were stored and executed whenever the affected page was loaded.
Impact	Persistent client-side attacks may compromise user sessions.
Recommendation	Validate stored input and implement Content Security Policy (CSP).
Status	Confirmed

Proof of Concept / Evidence:

Step-1: Open problem in DVWA.



Step-2: Try to inject xss basic payload.



6. Conclusion:

The assessment identified multiple critical and high-risk vulnerabilities within the DVWA environment. These findings demonstrate how insecure coding practices, insufficient validation, and weak security controls can expose applications to serious attacks. Applying secure coding standards, proper input validation, authentication controls, and layered security mechanisms will significantly reduce exploitation risk.